版本: **1.0.5　2024 年 03 月**

AnyCloud39AV100 NNE 使用示例

© 2024 Guangzhou Anyka Microelectronics Co., Ltd.　All rights reserved

声　　明

本手册的版权归广州安凯微电子股份有限公司所有，受相关法律法规的保护。未经广州安凯微电子股份有限公司的事先书面许可，任何人不得复制、传播、泄露、许可使用本手册的任何内容。

本手册所涉及的知识产权归广州安凯微电子股份有限公司所有(或经合作商授权许可使用)，任何人不得侵犯。

本手册不对包括但不限于下列事项担保：适销性、特殊用途的适用性；实施该用途不会侵害第三方的知识产权权利；等等。

广州安凯微电子股份有限公司不对客户等任何主体因使用本手册或执行本手册内容而带来/产生的任何直接及间接损害负责。

本手册是按当前的状态提供，仅供参考。如随附产品或本书内容有更改，恕不另行通知。

联　系　方　式

广州安凯微电子股份有限公司
地址：广州市黄埔区知识城博文路 107 号安凯微电子 H 大厦
电话: (86)-20-3221 9000
传真: (86)-20-3221 9258
邮编: 510555

销售热线:
(86)-20-3221 9499

电子邮箱:
sales@anyka.com

主页:
http: //www. anyka.com

Anyka Confidential For Giec Use Only

版 本 变 更 说 明

以下表格对于本文档的版本变更做一个简要的说明。版本变更仅限于技术内容的变更，不包括版式、格式、句法等的变更。

版本	说明	完成日期
V1.0.0	正式发布	2022 年 11 月
V1.0.1	增加 ssd_slim320 网络使用 NNE_QUAN_METHOD_USR_SET 量化方式的介绍。	2022 年 12 月
V1.0.2	增加 Demo：op_concat_updample.onnx。用来验证 concat 和 upsample 算子。	2023 年 02 月
V1.0.3	对 Demo 目录下增加的内容加以描述。 上一版本描述在 visual studio 工具中的操作步骤，当前版本修改为描述 linux 环境下的操作步骤。	2023 年 04 月
V1.0.4	删除 onet。修改 ssd_mobilenetv1 的输入分辨率。 增加 mobilenet_v2、resnet_v2_50、resnet20_cifar10、yolov5_s、yolov8_n、yolox_nano 网络。	2023 年 11 月
V1.0.5	增加 yolov5n。	2024 年 03 月

Anyka Confidential For Giec Use Only

目录

1 概述 7
2 Demo1: mobilenet_v1 9
2.1 来源 9
2.2 步骤 9
2.2.1 准备测试/校准数据集 9
2.2.2 填写配置文件 9
2.2.3 转换模型为 onnx 10
2.2.4 写后处理 C 代码 10
2.2.5 convert 10
2.2.6 quan 10
2.2.7 check 11
2.2.8 simulate 11
2.2.9 上板验证 11
3 Demo2: mobilenet_v2_tf 12
3.1 来源 12
3.2 步骤 12
3.2.1 准备测试/校准数据集 12
3.2.2 填写配置文件 12
3.2.3 转换模型为 onnx 13
3.2.4 写后处理 C 代码 13
3.2.5 convert 13
3.2.6 quan 13
3.2.7 check 14
3.2.8 simulate 14
3.2.9 上板验证 14
4 Demo3: resnet_v2_50_retrain 15
4.1 来源 15
4.2 步骤 15
4.2.1 准备测试/校准数据集 15
4.2.2 填写配置文件 15
4.2.3 转换模型为 onnx 16
4.2.4 写后处理 C 代码 16
4.2.5 convert 16
4.2.6 quan 16
4.2.7 check 17
4.2.8 simulate 17
4.2.9 上板验证 18
5 Demo4: resnet20_cifar10 18
5.1 来源 18
5.2 步骤 18
5.2.1 准备测试/校准数据集 18
5.2.2 填写配置文件 19

Anyka Confidential For Ciec Use Only

5.2.3 转换模型为 onnx 19
5.2.4 写后处理 C 代码 19
5.2.5 convert 19
5.2.6 quan 20
5.2.7 check 20
5.2.8 simulate 20
5.2.9 上板验证 21
6 Demo5: shufflenet_v2 21
6.1 来源 21
6.2 步骤 21
6.2.1 准备测试/校准数据集 21
6.2.2 填写配置文件 21
6.2.3 转换模型为 onnx 22
6.2.4 写后处理 C 代码 22
6.2.5 convert 22
6.2.6 quan 23
6.2.7 check 23
6.2.8 simulate 23
6.2.9 编译 24
7 Demo6: ssd_mobilenetv1 24
7.1 来源 24
7.2 步骤 24
7.2.1 准备测试/校准数据集 24
7.2.2 填写配置文件 25
7.2.3 转换模型为 onnx 25
7.2.4 写后处理 C 代码 25
7.2.5 convert 26
7.2.6 quan 26
7.2.7 check 26
7.2.8 simulate 27
7.2.9 编译 27
8 Demo7: ssd_slim320 27
8.1 来源 27
8.2 步骤 28
8.2.1 准备测试/校准数据集 28
8.2.2 填写配置文件 28
8.2.3 转换模型为 onnx 28
8.2.4 写后处理 C 代码 29
8.2.5 convert 29
8.2.6 quan 29
8.2.7 check 29
8.2.8 simulate 30
8.2.9 编译 30
9 Demo8: yolov5_s_retrain 30

Anyka Confidential For Giec Use Only

9.1 来源 30
9.2 步骤 31
9.2.1 准备测试/校准数据集 31
9.2.2 填写配置文件 31
9.2.3 转换模型为 onnx 31
9.2.4 写后处理 C 代码 31
9.2.5 convert 32
9.2.6 quan 32
9.2.7 check 32
9.2.8 simulate 32
9.2.9 编译 33
10 Demo9: yolov8_n_retrain 33
10.1 来源 33
10.2 步骤 33
10.2.1 准备测试/校准数据集 33
10.2.2 填写配置文件 34
10.2.3 转换模型为 onnx 34
10.2.4 写后处理 C 代码 34
10.2.5 convert 34
10.2.6 quan 35
10.2.7 check 35
10.2.8 simulate 35
10.2.9 编译 36
11 Demo10: yolox_nano_retrain 36
11.1 来源 36
11.2 步骤 36
11.2.1 准备测试/校准数据集 36
11.2.2 填写配置文件 36
11.2.3 转换模型为 onnx 37
11.2.4 写后处理 C 代码 37
11.2.5 convert 37
11.2.6 quan 38
11.2.7 check 38
11.2.8 simulate 38
11.2.9 编译 38
12 Demo11: yolov5_n_retrain 39
12.1 来源 39
12.2 步骤 39
12.2.1 准备测试/校准数据集 39
12.2.2 填写配置文件 39
12.2.3 转换模型为 onnx 40
12.2.4 写后处理 C 代码 40
12.2.5 convert 40
12.2.6 quan 40

Anyka Confidential For Giec Use Only

12.2.7 check.....41
12.2.8 simulate.....41
12.2.9 编译.....41

Anyka Confidential For Giec Use Only

1 概述

本文档介绍 NNE（Neural Network Engine）网络移植 Demo 用作参考。

PC 端工具环境参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6 节。上板运行环境参考《AnyCloud39AV100 NNE 网络移植编程手册》3.11 节。

Demo 列表：

Demo	名称	说明
Demo1	mobilenet_v1	轻量级分类（识别）网络。
Demo2	mobilenet_v2_tf	轻量级分类（识别）网络。
Demo3	resnet_v2_50_retrain	分类（识别）网络。
Demo4	resnet20_cifar10	轻量级分类（识别）网络。
Demo5	shufflenet_v2	轻量级分类（识别）网络。
Demo6	ssd_mobilenetv1	ssd 系列目标检测网络。
Demo7	ssd_slim320	ssd 系列目标检测网络。
Demo8	yolov5_s_retrain	yolo 系列目标检测网络。
Demo9	yolov8_n_retrain	yolo 系列目标检测网络。
Demo10	yolox_nano_retrain	yolo 系列目标检测网络。
Demo11	yolov5_n_retranin	yolo 系列目标检测网络。

onnx 文件说明。

Demo	模型	format	producer	imports
Demo1	mobilenet_v1.onnx	ONNX v6	tf2onnx 1.9.1	ai.onnx v11
Demo2	mobilenet_v2_1.0_224.onnx	ONNX v7	tf2onnx 1.14.0 None	ai.onnx v13, ai.onnx.ml v2
Demo3	resnet_v2_50_final_sim.onnx	ONNX v8	pytorch 2.0.1	ai.onnx v15
Demo4	resnet20-12fca82f.onnx	ONNX v5	pytorch 1.11.0	ai.onnx v10
Demo5	shufflenet_v2.onnx	ONNX v6	pytorch 1.6	ai.onnx v10
Demo6	ssd_mobilenetv1_300.onnx	ONNX v6	pytorch 1.6	ai.onnx v9
Demo7	ssd_slim320.onnx	ONNX v6	pytorch 1.6	ai.onnx v9
Demo8	yolo_v5_s_0915_3outputs_sim.onnx	ONNX v5	onnx.utils.extract_model	ai.onnx v10
Demo9	yolov8n_6outputs.onnx	ONNX v5	onnx.utils.extract_model	ai.onnx v10
Demo10	yolox_nano.onnx	ONNX v6	onnx.utils.extract_model	ai.onnx v11
Demo11	yolov5n.onnx	ONNX v6	onnx.utils.extract_model	ai.onnx v12

以上 Demo 中，Demo1-Demo5 是分类网络，Demo6-Demo11 是检测网络。Demo3、Demo8、Demo9、Demo10、Demo11 网络结构在开源模型基础上修改，将 NNE_V1 不支持的算子用

Anyka Confidential For Ciec Use Only

其他算子替换，且重新训练得到新的参数。其他 Demo 使用开源的预训练模型未经修改。

为了方便描述，下文中的文档结构根目录记为“nne_compile_tool”，文档结构为如图 1-1。

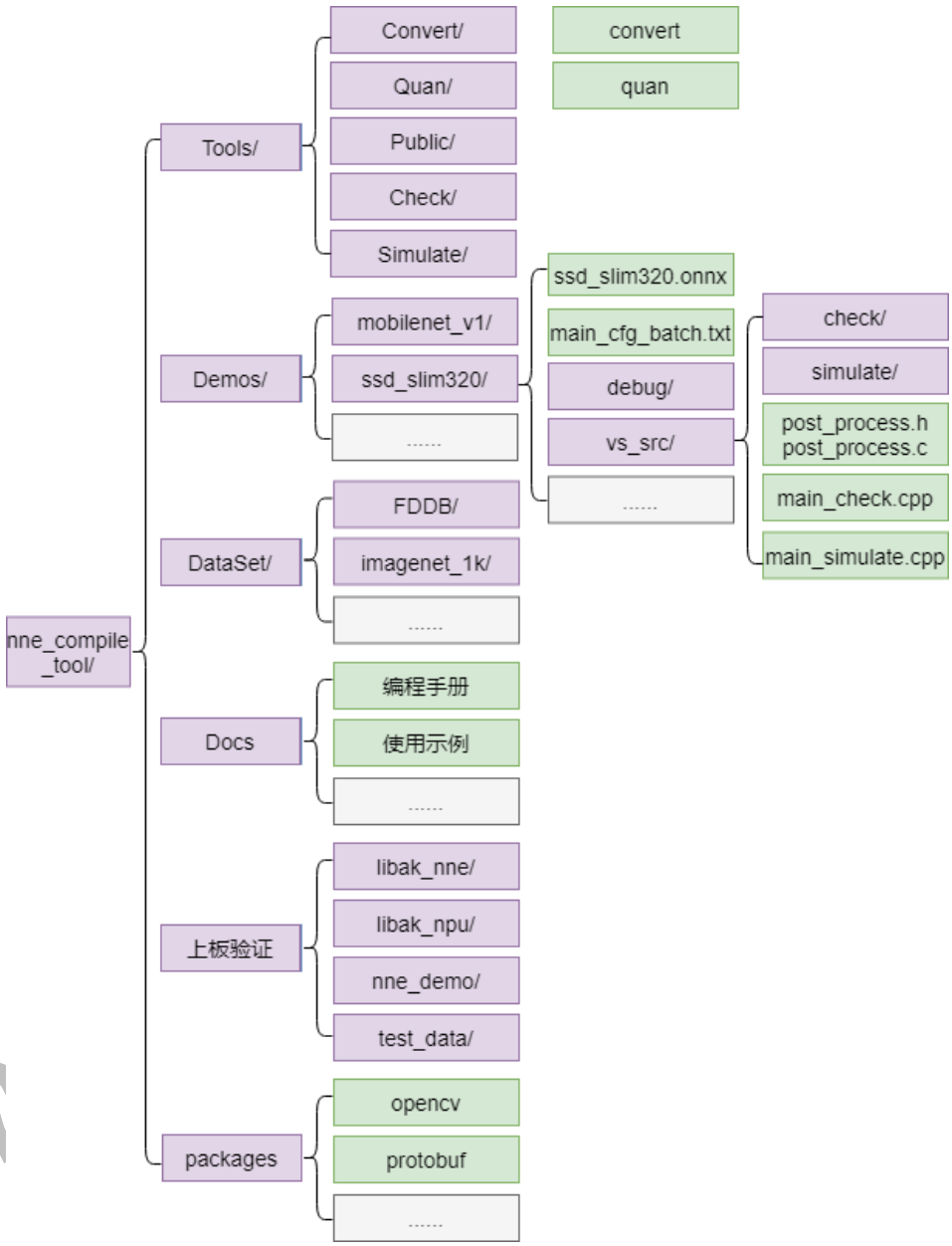

图 1-1 工具包文件结构图

nne_compile_tool/Tools/目录下存放 PC 端工具，包含 4 个步骤，分别是 Convert、Quan、Check、Simulate。Convert 目录下 convert 是可执行文件，将 onnx 模型转换为特定浮点模型；Quan 目录下 quan 是可执行文件，将浮点模型转换为定点模型；Check 路径下是浮点模型和定点模型正向推理需要的部分源码；Simulate 路径下是定点模型仿真需要的部分源码；Public 中的文件是四个步骤的公共文件。nne_compile_tool/Tools /目录下的文件是所有网络通用的文件，与具体网络无关。与网络相关的后处理函数及 check 步骤主函数、simulate 步骤主函数存放在 nne_compile_tool/Demos/xxx/vs_src/目录下。

nne_compile_tool/Demos/目录下存放了多个 demo，用来演示网络移植步骤。每个 demo 包含 onnx 文件、配置文件、后处理 c 文件、check 步骤主函数、simulate 步骤主函数等。

nne_compile_tool/Demos/vs_src/目录下已将 check 和 simulate 步骤所需文件放在一起且已编译成库可直接使用。

2 Demo1: mobilenet_v1

2.1 来源

模型下载地址：

https://github.com/tensorflow/models/tree/master/research/slim

http://download.tensorflow.org/models/mobilenet_v1_2018_02_22/mobilenet_v1_1.0_224.tgz

demo 使用模型：mobilenet_v1_1.0_224.tflite

2.2 步骤

2.2.1 准备测试/校准数据集

数据集：nne_compile_tool/DataSet/imagenet_1k/224_224_png/，已经经过 crop/resize 等预处理，分辨率均为 224*224*3。图片名列表文件：nne_compile_tool/DataSet/imagenet_1k/224_224_png/filename_list.txt。

2.2.2 填写配置文件

在 nne_compile_tool/Demos/mobilenet_v1/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/mobilenet_v1/
onnx_file	mobilenetv1.onnx
input_w	224
input_h	224
input_c	3
input_color	NNE_INPUT_COLOR_RGB
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	mobilenet_v1_float_model.bin
task	NNE_TASK_Classification
dataset_dir	nne_compile_tool/DataSet/imagenet_1k/224_224_png/
dataset_list	nne_compile_tool/DataSet/imagenet_1k/224_224_png/filename_list.txt
quan_image_num	100
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.001
quan_fixed_model	mobilenet_v1_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	1000
draw_result	AK_FALSE
image_bmp	debug/n01443537_ILSVRC2012_val_00048840.bmp
image_bin	debug/rgbline_mobilenetv1.bin
pool_adjust	0

Anyka Confidential For Gieo Use Only

model_header	0 0 0 0

2.2.3 转换模型为 onnx

以下步骤供参考：

在 python 环境中安装以下库

```
Pip install tensorflow
Pip install onnxruntime
Pip install -u tf2onnx
```

下载模型文件，在 cmd 中 cd 到保存 mobilenetv1 模型文件的根目录下。激活 python 环境，根据保存模型的具体类型分别进行 tf2onnx 操作，具体如下：

(1)当以 pb 文件作为输入时：

```
Python -m tf2onnx.convert --input mobilenet_v1_1.0_224_frozen.pb --output mobilenetv1.onnx
--inputs input:0 --outputs MobilenetV1/Predictions/Reshape_1:0
```

(2)当以 tflite 文件作为输入时：

```
python -m tf2onnx.convert --opset 13 --tflite mobilenet_v1_1.0_224.tflite --output
mobilenetv1.onnx
```

本文使用第二种方法。

2.2.4 写后处理 C 代码

该网络中 softmax 步骤作为后处理步骤。nne_compile_tool/Demos/mobilenet_v1/vs_src/目录下已提供：ak_nne_post_process.h 和 ak_nne_post_process.c。

2.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/mobilenet_v1/
../../Tools/Convert/convert  main_cfg_batch.txt
```

运行结束后在 nne_compile_tool/Demos/mobilenet_v1/目录下生成以下文件：

```
mobilenet_v1_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

2.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/mobilenet_v1/
../../Tools/Quan/quan  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/mobilenet_v1/目录下生成以下文件：

```
mobilenetv1_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
```

Anyka Confidential For Client Use Only

```
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

2.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/mobilenet_v1/
./vs_src/check  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/mobilenet_v1/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt）。

其他说明：

nne_compile_tool/Demos/mobilenet_v1/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

nne_compile_tool/Demos/mobilenet_v1/vs_src/目录下 main_accuracy.cpp 可作为精度统计代码，对这两个文件进行统计，浮点模型分类准确率为 0.742，定点模型分类准确率为 0.711.

该网络模型中包含激活函数“Relu6”，在浮点推理中不变，定点推理中使用的是“Relu”。

开源项目使用的图片预处理方式是 y=(x/255-mean)/std，其中 mean=[0.485, 0.456, 0.406]，std=[0.229, 0.224, 0.225]。在此 demo 中改为 y=(x-128)/128。

2.2.8 simulate

模拟开发板接口格式跑定点正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/mobilenet_v1/
./vs_src/simulate  main_cfg_batch.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 mobilenet_v1_224.rgbline，n01443537_ILSVRC2012_val_00048840_box.txt。

其他说明：

nne_compile_tool/Demos/mobilenet_v1/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

2.2.9 上板验证

2.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

2.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt

Anyka Confidential For Giec Use Only

目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
mobilenetv1_fixed_model.bin	nne_compile_tool/Demos/mobilenet_v1/ 或者 nne_compile_tool/上板验证/test_data/mobilenet_v1/
mobilenet_v1_224.rgbline	nne_compile_tool/Demos/mobilenet_v1/debug/ 或者 nne_compile_tool/上板验证/test_data/mobilenet_v1/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo mobilenet_v1  mobilenet_v1_fixed_model.bin  mobilenet_v1_224.rgbline  224 224 3
```

打印结果与 nne_compile_tool/上板验证/test_data/mobilenet_v1/result.txt 比对验证正确性。

3 Demo2: mobilenet_v2_tf

3.1 来源

模型下载地址：

https://github.com/tensorflow/models/tree/master/research/slim

https://storage.googleapis.com/mobilenet_v2/checkpoints/mobilenet_v2_1.4_224.tgz

demo 使用模型：mobilenet_v2_1.0_224.tflite

3.2 步骤

3.2.1 准备测试/校准数据集

数据集：nne_compile_tool/DataSet/imagenet_1k/224_224_png/，已经经过 crop/resize 等预处理，分辨率均为 224*224*3。图片名列表文件：nne_compile_tool/DataSet/imagenet_1k/224_224_png/filename_list.txt。

3.2.2 填写配置文件

在 nne_compile_tool/Demos/mobilenet_v2_tf/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/mobilenet_v2_tf/
onnx_file	mobilenet_v2_1.0_224.onnx
input_w	224
input_h	224
input_c	3
input_color	NNE_INPUT_COLOR_RGB
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	mobilenet_v2_tf_float_model.bin
task	NNE_TASK_Classification
dataset_dir	nne_compile_tool/DataSet/imagenet_1k/224_224_png/
dataset_list	nne_compile_tool/DataSet/imagenet_1k/224_224_png/filename_list.txt
quan_image_num	100

Anyka Confidential For Giec Use Only

quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.0005
quan_fixed_model	mobilenet_v2_tf_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	1000
draw_result	AK_FALSE
image_bmp	debug/n01443537_ILSVRC2012_val_00048840.bmp
image_bin	debug/mobilenet_v2_224.rgbline
pool_adjust	0
model_header	0 0 0 0

3.2.3 转换模型为 onnx

以下步骤供参考：

在 python 环境中安装以下库

```
Pip install tensorflow
Pip install onnxruntime
Pip install -u tf2onnx
```

下载模型文件，在 cmd 中 cd 到保存 mobilenet_v2 模型文件的根目录下。激活 python 环境，根据保存模型的具体类型分别进行 tf2onnx 操作，具体如下：

当以 tflite 文件作为输入时：

```
python -m tf2onnx.convert --opset 13 --tflite mobilenet_v2_1.0_224.tflite  --output
mobilenet_v2_1.0_224.onnx
```

3.2.4 写后处理 C 代码

该网络中 softmax 步骤作为后处理步骤。nne_compile_tool/Demos/mobilenet_v2_tf/vs_src/目录下已提供：ak_nne_post_process.h 和 ak_nne_post_process.c。

3.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/mobilenet_v2_tf/
../../Tools/Convert/convert    main_cfg_batch.txt
```

运行结束后在 nne_compile_tool/Demos/mobilenet_v2_tf/目录下生成以下文件：

```
mobilenet_v2_tf_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

3.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/mobilenet_v2_tf/
```

Anyka Confidential For Giec Use Only

```
../../Tools/Quan/quan  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/mobilenet_v2_tf/目录下生成以下文件：

```
mobilenet_v2_tf_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

3.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/mobilenet_v2_tf/
./vs_src/check  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/mobilenet_v2_tf/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt）。

其他说明：

nne_compile_tool/Demos/mobilenet_v2_tf/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

nne_compile_tool/Demos/mobilenet_v2_tf/vs_src/目录下 main_accuracy.cpp 可作为精度统计代码，对这两个文件进行统计，浮点模型分类准确率为 0.732，定点模型分类准确率为 0.7.

该网络模型中包含激活函数“Relu6”，在浮点推理中不变，定点推理中使用的是“Relu”。

开源项目使用的图片预处理方式是 y=(x/255-mean)/std，其中 mean=[0.485, 0.456, 0.406]，std=[0.229, 0.224, 0.225]。在此 demo 中改为 y=(x-128)/128。

3.2.8 simulate

模拟开发板接口格式跑定点正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/mobilenet_v2_tf/
./vs_src/simulate  main_cfg_batch.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 mobilenet_v2_224.rgbline，result.txt。

其他说明：

nne_compile_tool/Demos/mobilenet_v2_tf/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

3.2.9 上板验证

3.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到

Anyka Confidential For Giec Use Only

可执行文件：nne_demo。

3.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
mobilenet_v2_tf_fixed_model.bin	nne_compile_tool/Demos/mobilenet_v2_tf/ 或者 nne_compile_tool/上板验证/test_data/mobilenet_v2_tf/
mobilenet_v2_224.rgbline	nne_compile_tool/Demos/mobilenet_v2_tf/debug/ 或者 nne_compile_tool/上板验证/test_data/mobilenet_v2_tf/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo mobilenet_v2_tf mobilenet_v2_tf_fixed_model.bin mobilenet_v2_224.rgbline 224 224 3
```

打印结果与 nne_compile_tool/上板验证/test_data/mobilenet_v2_tf/result.txt 比对验证正确性。

4 Demo3: resnet_v2_50_retrain

4.1 来源

原始工程来自于 github 开源项目 [pytorch-cifar100](github.com/weiaicunzai/pytorch-cifar100)，在此工程基础上修改网络结构以适应 AV100-NNE，并基于 ImageNet 重新训练，得到预训练模型。详细情况请参考训练工程。

4.2 步骤

4.2.1 准备测试/校准数据集

数据集：nne_compile_tool/DataSet/imagenet_1k/224_224_png/，已经经过 crop/resize 等预处理，分辨率均为 224*224*3。图片名列表文件：nne_compile_tool/DataSet/imagenet_1k/224_224_png/filename_list.txt。

4.2.2 填写配置文件

在 nne_compile_tool/Demos/resnet_v2_50_retrain/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/resnet_v2_50_retrain/
onnx_file	resnet_v2_50_final_sim.onnx
input_w	224
input_h	224
input_c	3
input_color	NNE_INPUT_COLOR_RGB

Anyka Confidential For Giec Use Only

input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	resnet_v2_50_Imagenet_float_model.bin
task	NNE_TASK_Classification
dataset_dir	nne_compile_tool/DataSet/imagenet_1k/224_224_png/
dataset_list	nne_compile_tool/DataSet/imagenet_1k/224_224_png/filename_list.txt
quan_image_num	100
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.001
quan_fixed_model	resnet_v2_50_Imagenet_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	1000
draw_result	AK_FALSE
image_bmp	debug/n01443537_ILSVRC2012_val_00048840.bmp
image_bin	debug/resnet_v2_50_Imagenet_224.rgbline
pool_adjust	0
model_header	0 0 0 0

经过量化分析后调整了量化系数，重新填写配置文件 main_cfg_UsrSet.txt。

用户请直接使用 main_cfg_UsrSet.txt。

4.2.3 转换模型为 onnx

训练工程中提供 pytorch 转 onnx 的脚本。一般情况下 pytorch 转 onnx 可调用“torch.onnx.export”。如果生成的 onnx 包含 softmax 且在网络最后部分，可调用“onnx.utils.extract_model”提取 onnx 的子模型，将 softmax 等算子排除在外。

4.2.4 写后处理 C 代码

该网络中 softmax 步骤作为后处理步骤。nne_compile_tool/Demos/resnet_v2_50_retrain/vs_src/目录下已提供：ak_nne_post_process.h 和 ak_nne_post_process.c。

4.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/resnet_v2_50_retrain/
../../Tools/Convert/convert    main_cfg_UsrSet.txt
```

运行结束后在 nne_compile_tool/Demos/resnet_v2_50_retrain/目录下生成以下文件：

```
resnet_v2_50_Imagenet_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

4.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

Anyka Confidential For Giec Use Only

```
cd nne_compile_tool/Demos/resnet_v2_50_retrain/
../../Tools/Quan/quan    main_cfg_UsrSet.txt
```

运行结束后，在 nne_compile_tool/Demos/resnet_v2_50_retrain/目录下生成以下文件：

```
resnet_v2_50_Imagenet_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

分析了网络结构（quan_output_network_info.c）和量化统计结果（quan_output_HistTH_feature_shift.txt），决定对 FC 层 feature_shift 进行调整。FC 层拆分成 [Split_C+Conv+Conv+Add]，其中 Add 又拆分成 [Build_Concat_Group+Conv+Conv+Concat_C]，FC 层总共拆分为 7 层。将 blob_idx 为[260-266]这 7 个 blob 的 shift 改为 3，因为[260-266]这 7 个 blob 是 FC 层做完 conv 之后的数据块，此时最大值是最重要的，不能让最大值 clip 掉。

后续步骤请用户使用 main_cfg_UsrSet.txt。

4.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/resnet_v2_50_retrain/
./vs_src/check    main_cfg_UsrSet.txt
```

运行结束后，在 nne_compile_tool/Demos/resnet_v2_50_retrain/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt）。

其他说明：

nne_compile_tool/Demos/resnet_v2_50_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

nne_compile_tool/Demos/resnet_v2_50_retrain/vs_src/目录下 main_accuracy.cpp 可作为精度统计代码，对这两个文件进行统计，浮点模型分类准确率为 0.742，定点模型分类准确率为 0.742.

训练与推理使用的预处理方式相同，y=(x-128)/128。

4.2.8 simulate

模拟开发板接口格式跑定点正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/resnet_v2_50_retrain/
./vs_src/simulate    main_cfg_UsrSet.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 resnet_v2_50_Imagenet_224.rgbline，result.txt。

Anyka Confidential For Ciec Use Only

其他说明：

nne_compile_tool/Demos/resnet_v2_50_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

4.2.9 上板验证

4.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

4.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
resnet_v2_50_Imagenet_fixed_model.bin	nne_compile_tool/Demos/resnet_v2_50_retrain/ 或者 nne_compile_tool/上板验证/test_data/resnet_v2_50_retrain/
resnet_v2_50_Imagenet_224.rgbline	nne_compile_tool/Demos/resnet_v2_50_retrain/debug/ 或者 nne_compile_tool/上板验证/test_data/resnet_v2_50_retrain/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo resnet_v2_50_retrain resnet_v2_50_Imagenet_fixed_model.bin
resnet_v2_50_Imagenet_224.rgbline 224 224 3
```

打印结果与 nne_compile_tool/上板验证/test_data/resnet_v2_50_retrain/result.txt 比对验证正确性。

5 Demo4: resnet20_cifar10

5.1 来源

开源工程下载地址：

https://github.com/akamaster/pytorch_resnet_cifar10

5.2 步骤

5.2.1 准备测试/校准数据集

数据集：nne_compile_tool/DataSet/CIFAR-10/test/，已经从原始数据包中提取出 png 图片，分辨率均为 32*2*3。图片名列表文件：nne_compile_tool/DataSet/CIFAR-10/test/test_file_list.txt。提取脚本参考 nne_compile_tool/Demos/resnet20_cifar10/script/get_dataset.py

Anyka Confidential For Giec Use Only

5.2.2 填写配置文件

在 nne_compile_tool/Demos/resnet20_cifar10/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/resnet20_cifar10/
onnx_file	resnet20-12fca82f.onnx
input_w	32
input_h	32
input_c	3
input_color	NNE_INPUT_COLOR_RGB
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	resnet20_float_model.bin
task	NNE_TASK_Classification
dataset_dir	nne_compile_tool/DataSet/CIFAR-10/test/
dataset_list	nne_compile_tool/DataSet/CIFAR-10/test/test_file_list.txt
quan_image_num	1000
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.001
quan_fixed_model	resnet20_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	0
draw_result	AK_FALSE
image_bmp	debug/input_32_32_3.bmp
image_bin	debug/resnet20_cifar10_32.rgbline
pool_adjust	0
model_header	0 0 0 0

5.2.3 转换模型为 onnx

转换脚本参考 nne_compile_tool/Demos/resnet20_cifar10/script/convert_to_onnx.py

5.2.4 写后处理 C 代码

该网络中 softmax 步骤作为后处理步骤。nne_compile_tool/Demos/resnet20_cifar10/vs_src/目录下已提供：ak_nne_post_process.h 和 ak_nne_post_process.c。

5.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/resnet20_cifar10/
../../Tools/Convert/convert    main_cfg_batch.txt
```

运行结束后在 nne_compile_tool/Demos/resnet20_cifar10/目录下生成以下文件：

```
resnet20_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
```

Anyka Confidential For Giec Use Only

```
convert_output_net.txt
convert_output_net_analyse.txt
```

5.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/resnet20_cifar10/
../../Tools/Quan/quan  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/resnet20_cifar10/目录下生成以下文件：

```
resnet20_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

5.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/resnet20_cifar10/
./vs_src/check  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/resnet20_cifar10/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt）。

其他说明：

nne_compile_tool/Demos/resnet20_cifar10/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

nne_compile_tool/Demos/resnet20_cifar10/vs_src/目录下 main_accuracy.cpp 可作为精度统计代码，对这两个文件进行统计，浮点模型分类准确率为 0.765，定点模型分类准确率为 0.7581.

开源项目使用的图片预处理方式是 y=(x/255-mean)/std，其中 mean=[0.485, 0.456, 0.406]，std=[0.229, 0.224, 0.225]。在此 demo 中改为 y=(x-128)/128。

5.2.8 simulate

模拟开发板接口格式跑定点正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/resnet20_cifar10/
./vs_src/simulate  main_cfg_batch.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 resnet20_cifar10_32.rgbline，result.txt。

其他说明：

Anyka Confidential For Giec Use Only

nne_compile_tool/Demos/resnet20_cifar10/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

5.2.9 上板验证

5.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

5.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
resnet20_fixed_model.bin	nne_compile_tool/Demos/resnet20_cifar10/ 或者 nne_compile_tool/上板验证/test_data/resnet20_cifar10/
resnet20_cifar10_32.rgbline	nne_compile_tool/Demos/resnet20_cifar10/debug/ 或者 nne_compile_tool/上板验证/test_data/resnet20_cifar10/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo resnet20_cifar10 resnet20_fixed_model.bin resnet20_cifar10_32.rgbline 32 32 3
```

打印结果与 nne_compile_tool/上板验证/test_data/resnet20_cifar10/result.txt 比对验证正确性。

6 Demo5: shufflenet_v2

6.1 来源

开源项目地址：https://github.com/ericsun99/Shufflenet-v2-Pytorch
demo 使用模型：shufflenetv2_x1_69.390_88.412.pth.tar

6.2 步骤

6.2.1 准备测试/校准数据集

数据集：nne_compile_tool/DataSet/imagenet_1k/224_224_png/，已经经过 crop/resize 等预处理，分辨率均为 224*224*3。图片名列表文件：nne_compile_tool/DataSet/imagenet_1k/224_224_png/filename_list.txt。

6.2.2 填写配置文件

在 nne_compile_tool/Demos/shufflenet_v2/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/shufflenet_v2/
onnx_file	shufflenet_v2.onnx

Anyka Confidential For Giec Use Only

input_w	224
input_h	224
input_c	3
input_color	NNE_INPUT_COLOR_RGB
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	shufflenet_v2_float_model.bin
task	NNE_TASK_Classification
dataset_dir	nne_compile_tool/DataSet/imagenet_1k/224_224_png/
dataset_list	nne_compile_tool/DataSet/imagenet_1k/224_224_png/filename_list.txt
quan_image_num	100
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.001
quan_fixed_model	shufflenet_v2_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	1000
draw_result	AK_FALSE
image_bmp	debug/n01443537_ILSVRC2012_val_00048840.bmp
image_bin	debug/shufflenet_v2_224.rgbline
pool_adjust	1
model_header	0 0 0 0

注意，此处 pool_adjust 设置为 1，模型开头的“MaxPool”会和前面的“conv+bn+relu”组合为“基本级联层”，由 NNE 实现。MaxPool 的输出结果会和 pytorch 推理结果有差异，NNE 仅支持 Pooling 的 Pad 加在“右边”和“底边”，而 pytorch 推理时 pad 加在“左边”和“上边”，因此导致计算结果差异。如果要求计算结果完全一致，则将 pool_adjust 设置为 0，此时“MaxPool”不会和前面的“conv+bn+relu”组合为“基本级联层”，而是单独由 CPU 运算，耗时增加。

6.2.3 转换模型为 onnx

以下步骤供参考：

- 将开源项目保存到本地，路径为 proj_dir = x:/Shufflenet-v2-Pytorch
- 将 nne_compile_tool/Demos/shufflenet_v2/script/convert_to_onnx.py 和 ShuffleNetV2.py 复制到 proj_dir 路径下（原文件请备份）。
- 运行 convert_to_onnx.py。运行结束后在 proj_dir/models/my_onn/x 路径下生成文件 shufflenetv2_x1_tmp1.onnx。将 shufflenetv2_x1_tmp1.onnx 复制到 nne_compile_tool/Demos/shufflenet_v2/目录下。

6.2.4 写后处理 C 代码

该网络中 softmax 步骤作为后处理步骤。nne_compile_tool/Demos/shufflenet_v2/vs_src/目录下已提供 C 代码示例 ak_nne_post_process.h 和 ak_nne_post_process.c。

6.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/shufflenet_v2/
```

Anyka Confidential For Giec Use Only

```
../../Tools/Convert/convert    main_cfg_batch.txt
```

运行结束后在 nne_compile_tool/Demos/shufflenet_v2/目录下生成以下文件：

```
shufflenet_v2_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

6.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/shufflenet_v2/
../../Tools/Quan/quan    main_cfg_batch.txt
```

运行结束后，在/nne_compile_tool/Demos/shufflenet_v2/目录下生成以下文件：

```
shufflenet_v2_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

6.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/shufflenet_v2/
./vs_src/check    main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/shufflenet_v2/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt）。

其他说明：

nne_compile_tool/Demos/shufflenet_v2/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

nne_compile_tool/Demos/shufflenet_v2/vs_src/目录下 main_accuracy.cpp 可作为精度统计代码，对这两个文件进行统计，浮点模型分类准确率为 0.619，定点模型分类准确率为 0.596。配置文件中 pool_adjust 的不同设置会产生不同精度。如果设置为 0，则浮点推理精度为 0.621，如果设置为 1，则浮点推理精度为 0.619。

开源项目使用的图片预处理方式是 y=(x/255-mean)/std，其中 mean=[0.485, 0.456, 0.406]，std=[0.229, 0.224, 0.225]。在此 demo 中改为 y=(x-128)/128。

6.2.8 simulate

模拟开发板接口格式跑定点正向推理。

Anyka Confidential For Giec Use Only

运行命令行：

```
cd nne_compile_tool/Demos/shufflenet_v2/
./vs_src/simulate  main_cfg_batch.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 shufflenet_v2_224.rgbline，n01443537_ILSVRC2012_val_00048840_box.txt。

其他说明：

nne_compile_tool/Demos/shufflenet_v2/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

6.2.9 编译

6.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

6.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
shufflenetv2_fixed_model.bin	nne_compile_tool/Demos/shufflenet_v2/ 或者 nne_compile_tool/上板验证/test_data/shufflenet_v2/
shufflenet_v2_224.rgbline	nne_compile_tool/Demos/shufflenet_v2/debug/ 或者 nne_compile_tool/上板验证/test_data/shufflenet_v2/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo shufflenet_v2 shufflenetv2_fixed_model.bin shufflenet_v2_224.rgbline 224 224 3
```

打印结果与 nne_compile_tool/上板验证/test_data/shufflenet_v2/n01443537_ILSVRC2012_val_00048840_box.txt 比对验证正确性。

7 Demo6: ssd_mobilenetv1

7.1 来源

开源项目地址：https://github.com/qfgaohao/pytorch-ssd
demo 使用模型：https://storage.googleapis.com/models-hao/mobilenet-v1-ssd-mp-0_675.pth

7.2 步骤

7.2.1 准备测试/校准数据集

使用 VOC2007 作为测试/校准数据集，该图片集中分辨率不一致，保持长宽比不变，缩放、填充到 300*300 尺寸，取其中 100 张图，保存路径：nne_compile_tool/DataSet/VOC2007/300_300_png/，图片名信息保存在

Anyka Confidential For Gies Use Only

nne_compile_tool/DataSet/VOC2007/300_300_png/file_list.txt。

7.2.2 填写配置文件

nne_compile_tool/Demos/ssd_mobilenetv1/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/ssd_mobilenetv1/
onnx_file	ssd_mobilenetv1_300.onnx
input_w	300
input_h	300
input_c	3
input_color	NNE_INPUT_COLOR_RGB
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	ssd_mobilenetv1_300_float_model.bin
task	NNE_TASK_Detection
dataset_dir	nne_compile_tool/DataSet/VOC2007/300_300_png/
dataset_list	nne_compile_tool/DataSet/VOC2007/300_300_png/file_list.txt
quan_image_num	10
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.0001
quan_fixed_model	ssd_mobilenetv1_300_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	100
draw_result	AK_TRUE
image_bmp	debug/000002.bmp
image_bin	debug/ssd_mobilenetv1_300.rgbline
pool_adjust	0
model_header	0 0 0 0

7.2.3 转换模型为 onnx

以下步骤供参考：

- 将开源项目保存到本地，路径为 proj_dir = x:/pytorch-ssd-master
- 将 nne_compile_tool/Demos/ssd_mobilenetv1/script/convert_to_onnx.py 复制到 proj_dir 路径下。
- 将 nne_compile_tool/Demos/ssd_mobilenetv1/script/ssd.py 复制到 proj_dir/vision/ssd/ 路径下并覆盖（原文件请备份）。
- 运行 convert_to_onnx.py。运行结束后在 proj_dir/models/my_onnx 路径下生成文件 ssd_mobilenetv1_300.onnx。将 ssd_mobilenetv1_300onnx 复制到 nne_compile_tool/Demos/ssd_mobilenetv1/目录下。

在生成 onnx 模型时，输出特征金字塔，而将针对特征金字塔的计算剔除，参考《编程手册》3.1 节的说明。

7.2.4 写后处理 C 代码

该网络中对特征金字塔做处理得出最终目标框的过程作为后处理步骤。C 代码示例保存

Anyka Confidential For Giec Use Only

在 nne_compile_tool/Demos/ssd_mobilenetv1/vs_src/目录下，ak_nne_post_process.h 和 ak_nne_post_process.c。

7.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/ssd_mobilenetv1/
../../Tools/Convert/convert  main_cfg_batch.txt
```

运行结束后在 F:/nne_compile_tool/Demos/ssd_mobilenetv1/目录下生成以下文件：

```
ssd_mobilenetv1_300_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

7.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/ssd_mobilenetv1/
../../Tools/Quan/quan  main_cfg_batch.txt
```

运行结束后，在/nne_compile_tool/Demos/ssd_mobilenetv1/目录下生成以下文件：

```
ssd_mobilenetv1_300_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

7.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/ssd_mobilenetv1/
./vs_src/check  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/ssd_mobilenetv1/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt），保存了画框后的图片。

其他说明：

nne_compile_tool/Demos/ssd_mobilenetv1/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

Anyka Confidential For Giec Use Only

7.2.8 simulate

模拟开发板接口格式跑定点正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/ssd_mobilenetv1/
./vs_src/simulate  main_cfg_batch.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 ssd_mobilenetv1_300.rgbline，vs_simulate_output.txt。

其他说明：

nne_compile_tool/Demos/ssd_mobilenetv1/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

7.2.9 编译

7.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

7.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
ssd_mobilenetv1_300_fixed_model.bin	nne_compile_tool/Demos/ssd_mobilenetv1/ 或者 nne_compile_tool/上板验证/test_data/ssd_mobilenetv1/
ssd_mobilenetv1_300.rgbline	nne_compile_tool/Demos/ssd_mobilenetv1/debug/ 或者 nne_compile_tool/上板验证/test_data/ssd_mobilenetv1/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo ssd_mobilenetv1 ssd_mobilenetv1_300_fixed_model.bin
ssd_mobilenetv1_300.rgbline 300 300 3
```

打印结果与 nne_compile_tool/上板验证/test_data/ssd_mobilenetv1/vs_simulate_output.txt 比对验证正确性。

8 Demo7: ssd_slim320

8.1 来源

开源项目地址：https://github.com/Linzaer/Ultra-Light-Fast-Generic-Face-Detector-1MB
demo 使用模型：./models/pretrained/version-slim-320.pth

Anyka Confidential For Specific Use Only

8.2 步骤

8.2.1 准备测试/校准数据集

使用 FDDB 作为测试/校准数据集，该图片集中分辨率不一致，保持长宽比不变，缩放、填充到 320*240 尺寸，预处理后数据集：nne_compile_tool/DataSet/FDDB/320_240_png/，图片名信息保存在 nne_compile_tool/DataSet/FDDB/320_240_png/filename_list.txt。

8.2.2 填写配置文件

在 nne_compile_tool/Demos/ssd_slim320/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/ssd_slim320/
onnx_file	version-slim-320.onnx
input_w	320
input_h	240
input_c	3
input_color	NNE_INPUT_COLOR_RGB
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	ssd_slim320_float_model.bin
task	NNE_TASK_Detection
dataset_dir	nne_compile_tool/DataSet/FDDB/320_240_png/
dataset_list	nne_compile_tool/DataSet/FDDB/320_240_png/filename_list.txt
quan_image_num	100
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.0001
quan_fixed_model	ssd_slim320_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	100
draw_result	AK_TRUE
image_bmp	debug/2002_07_24_big_img_518.bmp
image_bin	debug/ssd_slim320.rgbline
pool_adjust	0
model_header	0 0 0 0

8.2.3 转换模型为 onnx

以下步骤供参考：

- 将开源项目保存到本地，路径为 proj_dir = x:/Ultra-Light-Fast-Generic-Face-Detector-1MB-master
- 将 nne_compile_tool/Demos/ssd_slim320/script/convert_to_onnx.py 复制到 proj_dir 路径下。
- 将 nne_compile_tool/Demos/ssd_slim320/script/ssd.py 复制到 proj_dir/vision/ssd/ 路径下并覆盖（原文件请备份）。
- 运行 convert_to_onnx.py。运行结束后在 proj_dir/models/my_onnx 路径下生成文件 version-slim-320.onnx。将 version-slim-320.onnx 复制到

Anyka Confidential For Giec Use Only

nne_compile_tool/Demos/ssd_slim320/目录下。

在生成 onnx 模型时，输出特征金字塔，而将针对特征金字塔的计算剔除，参考《编程手册》3.1 节的说明。

8.2.4 写后处理 C 代码

该网络中对特征金字塔做处理得出最终目标框的过程作为后处理步骤。C 代码示例保存在 nne_compile_tool/Demos/ssd_slim320/vs_src/目录下，ak_nne_post_process.h 和 ak_nne_post_process.c。

8.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/ssd_slim320/
../../Tools/Convert/convert  main_cfg_batch.txt
```

运行结束后在 nne_compile_tool/Demos/ssd_slim320/目录下生成以下文件：

```
ssd_slim320_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

8.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/ssd_slim320/
../../Tools/Quan/quan  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/ssd_slim320/目录下生成以下文件：

```
ssd_slim320_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

8.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/ssd_slim320/
./vs_src/check  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/ssd_slim320/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt），保存了画框后的图片。

其他说明：

Anyka Confidential For Giec Use Only

nne_compile_tool/Demos/ssd_slim320/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

8.2.8 simulate

模拟开发板接口格式跑定点正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/ssd_slim320/
./vs_src/simulate  main_cfg_batch.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 ssd_slim320.rgbline，vs_simulate_output.txt。

其他说明：

nne_compile_tool/Demos/ssd_slim320/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

8.2.9 编译

8.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

8.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
ssd_slim320_fixed_model.bin	nne_compile_tool/Demos/ssd_slim320/ 或者 nne_compile_tool/上板验证/test_data/ssd_slim320/
ssd_slim320.rgbline	nne_compile_tool/Demos/ssd_slim320/debug/ 或者 nne_compile_tool/上板验证/test_data/ssd_slim320/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo ssd_slim320 ssd_slim320_fixed_model.bin  ssd_slim320.rgbline 320 240 3
```

打印结果与 nne_compile_tool/上板验证/test_data/ssd_slim320/vs_simulate_output.txt 比对验证正确性。

9 Demo8: yolov5_s_retrain

9.1 来源

原始工程来自于 github 开源项目 [yolov5](https://github.com/ultralytics/yolov5) ，在此工程基础上修改网络结构以适应 AV100-NNE，并基于 COCO-2017 数据集重新训练，得到预训练模型。详细情况请参考训练工程。

Anyka Confidential For Giec Use Only

9.2 步骤

9.2.1 准备测试/校准数据集

使用 COCO-2017 作为测试/校准数据集，该图片集中分辨率不一致，保持长宽比不变，缩放、填充到640*640尺寸，预处理后数据集：nne_compile_tool/DataSet/COCO2017/640_640/，图片名信息保存在 nne_compile_tool/DataSet/COCO2017/filename_list.txt。

9.2.2 填写配置文件

在 nne_compile_tool/Demos/yolov5_s_retrain/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/yolov5_s_retrain/
onnx_file	yolo_v5_s_0915_3outputs_sim.onnx
input_w	640
input_h	640
input_c	3
input_color	NNE_INPUT_COLOR_RGB
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	yolov5_s_float_model.bin
task	NNE_TASK_Detection
dataset_dir	nne_compile_tool/DataSet/COCO2017/640_640/
dataset_list	nne_compile_tool/DataSet/COCO2017/filename_list.txt
quan_image_num	10
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.001
quan_fixed_model	yolov5_s_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	100
draw_result	AK_TRUE
image_bmp	debug/input_640_640.bmp
image_bin	debug/input_640_640.rgbline
pool_adjust	0
model_header	0 0 0 0

9.2.3 转换模型为 onnx

训练工程中提供 pytorch 转 onnx 的脚本。一般情况下 pytorch 转 onnx 可调用“torch.onnx.export”。如果生成的 onnx 包含网络后处理，可调用“onnx.utils.extract_model”提取 onnx 的子模型，将后处理部分的算子排除在外，参考《编程手册》3.1 节的说明。

9.2.4 写后处理 C 代码

该网络中对特征金字塔做处理得出最终目标框的过程作为后处理步骤。C 代码示例保存在 nne_compile_tool/Demos/yolov5_s_retrain/vs_src/目录下，ak_nne_post_process.h 和 ak_nne_post_process.c。

Anyka Confidential For Giec Use Only

9.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/yolov5_s_retrain/
../../Tools/Convert/convert  main_cfg_batch.txt
```

运行结束后在 nne_compile_tool/Demos/yolov5_s_retrain/目录下生成以下文件：

```
yolov5_s_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

9.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/yolov5_s_retrain/
../../Tools/Quan/quan  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/yolov5_s_retrain/目录下生成以下文件：

```
yolov5_s_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

9.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/yolov5_s_retrain/
./vs_src/check  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/yolov5_s_retrain/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt），保存了画框后的图片。

其他说明：

nne_compile_tool/Demos/yolov5_s_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

9.2.8 simulate

模拟开发板接口格式跑定点正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/yolov5_s_retrain/
```

Anyka Confidential For Ciec Use Only

```
./vs_src/simulate    main_cfg_batch.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 input_640_640.rgbline，vs_simulate_output.txt。

其他说明：

nne_compile_tool/Demos/yolov5_s_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

9.2.9 编译

9.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

9.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
yolov5_s_fixed_model.bin	nne_compile_tool/Demos/yolov5_s_retrain/ 或者 nne_compile_tool/上板验证/test_data/yolov5_s_retrain/
input_640_640.rgbline	nne_compile_tool/Demos/yolov5_s_retrain/debug/ 或者 nne_compile_tool/上板验证/test_data/yolov5_s_retrain/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo yolov5_s_retrain yolov5_s_fixed_model.bin input_640_640.rgbline 640 640 3
```

打印结果与 nne_compile_tool/上板验证/test_data/yolov5_s_retrain/vs_simulate_output.txt 比对验证正确性。

10 Demo9: yolov8_n_retrain

10.1 来源

原始工程来自于 github 开源项目[yolov8](https://github.com/ultralytics/ultralytics)，在此工程基础上修改网络结构以适应 AV100-NNE，并基于 COCO-2017 数据集重新训练，得到预训练模型。详细情况请参考训练工程。

10.2 步骤

10.2.1 准备测试/校准数据集

使用 COCO-2017 作为测试/校准数据集，该图片集中分辨率不一致，保持长宽比不变，缩放、填充到 640*640 尺寸，预处理后数据集：nne_compile_tool/DataSet/COCO2017/640_640/，图片名信息保存在 nne_compile_tool/DataSet/COCO2017/filename_list.txt。

Anyka Confidential For Giec Use Only

10.2.2 填写配置文件

在 nne_compile_tool/Demos/yolov8_n_retrain/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/yolov8_n_retrain/
onnx_file	yolov8n_6outputs.onnx
input_w	640
input_h	640
input_c	3
input_color	NNE_INPUT_COLOR_RGB
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	yolov8_n_float_model.bin
task	NNE_TASK_Detection
dataset_dir	nne_compile_tool/DataSet/COCO2017/640_640/
dataset_list	nne_compile_tool/DataSet/COCO2017/filename_list.txt
quan_image_num	10
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.001
quan_fixed_model	yolov8_n_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	100
draw_result	AK_TRUE
image_bmp	debug/input_640_640.bmp
image_bin	debug/input_640_640.rgbline
pool_adjust	0
model_header	0 0 0 0

10.2.3 转换模型为 onnx

训练工程中提供 pytorch 转 onnx 的脚本。一般情况下 pytorch 转 onnx 可调用“torch.onnx.export”。如果生成的 onnx 包含网络后处理，可调用“onnx.utils.extract_model”提取 onnx 的子模型，将后处理部分的算子排除在外，参考《编程手册》3.1 节的说明。

10.2.4 写后处理 C 代码

该网络中对特征金字塔做处理得出最终目标框的过程作为后处理步骤。C 代码示例保存在 nne_compile_tool/Demos/yolov8_n_retrain/vs_src/目录下，ak_nne_post_process.h 和 ak_nne_post_process.c。

10.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/yolov8_n_retrain/
../../Tools/Convert/convert    main_cfg_batch.txt
```

运行结束后在 nne_compile_tool/Demos/yolov8_n_retrain/目录下生成以下文件：

yolov8_n_float_model.bin

Anyka Confidential For Gec Use Only

```
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

10.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/yolov8_n_retrain/
../../Tools/Quan/quan  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/yolov8_n_retrain/目录下生成以下文件：

```
yolov8_n_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

10.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/yolov8_n_fixed_model/
./vs_src/check  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/yolov8_n_retrain/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt），保存了画框后的图片。

其他说明：

nne_compile_tool/Demos/yolov8_n_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

10.2.8 simulate

模拟开发板接口格式跑定点正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/yolov8_n_fixed_model/
./vs_src/simulate  main_cfg_batch.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 input_640_640.rgbline，vs_simulate_output.txt。

其他说明：

nne_compile_tool/Demos/yolov8_n_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

Anyka Confidential For Gtec Use Only

10.2.9 编译

10.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

10.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
yolov8_n_fixed_model.bin	nne_compile_tool/Demos/yolov8_n_retrain/ 或者 nne_compile_tool/上板验证/test_data/yolov8_n_retrain/
input_640_640.rgbline	nne_compile_tool/Demos/yolov8_n_retrain/debug/ 或者 nne_compile_tool/上板验证/test_data/yolov8_n_retrain/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo yolov8_n_retrain yolov8_n_fixed_model.bin input_640_640.rgbline 640 640 3
```

打印结果与 nne_compile_tool/上板验证/test_data/yolov8_n_retrain/vs_simulate_output.txt 比对验证正确性。

11 Demo10: yolox_nano_retrain

11.1 来源

原始工程来自于 github 开源项目[yolox](https://github.com/Megvii-BaseDetection/YOLOX)，在此工程基础上修改网络结构以适应 AV100-NNE，并基于 COCO-2017 数据集重新训练，得到预训练模型。详细情况请参考训练工程。

11.2 步骤

11.2.1 准备测试/校准数据集

使用 COCO-2017 作为测试/校准数据集，该图片集中分辨率不一致，保持长宽比不变，缩放、填充到416*416尺寸，预处理后数据集：nne_compile_tool/DataSet/COCO2017/416_416/，图片名信息保存在 nne_compile_tool/DataSet/COCO2017/filename_list.txt。

11.2.2 填写配置文件

在 nne_compile_tool/Demos/yolox_nano_retrain/路径下已提供 main_cfg_batch.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/yolox_nano_retrain/
onnx_file	yolo_x_nano_9outputs.onnx
input_w	416
input_h	416

Anyka Confidential For Giec Use Only

input_c	3
input_color	NNE_INPUT_COLOR_BGR
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	yolox_nano_float_model.bin
task	NNE_TASK_Detection
dataset_dir	nne_compile_tool/DataSet/COCO2017/416_416/
dataset_list	nne_compile_tool/DataSet/COCO2017/filename_list.txt
quan_image_num	10
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.001
quan_fixed_model	yolox_nano_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	100
draw_result	AK_TRUE
image_bmp	debug/dog_416.bmp
image_bin	debug/yolox_nano_input_416_416.rgbline
pool_adjust	0
model_header	0 0 0 0

注意，input_color 设置为 NNE_INPUT_COLOR_BGR，与训练时的设置保持一致。

11.2.3 转换模型为 onnx

训练工程中提供 pytorch 转 onnx 的脚本。一般情况下 pytorch 转 onnx 可调用“torch.onnx.export”。如果生成的 onnx 包含网络后处理，可调用“onnx.utils.extract_model”提取 onnx 的子模型，将后处理部分的算子排除在外，参考《编程手册》3.1 节的说明。

11.2.4 写后处理 C 代码

该网络中对特征金字塔做处理得出最终目标框的过程作为后处理步骤。C 代码示例保存在 nne_compile_tool/Demos/yolox_nano_retrain/vs_src/目录下，ak_nne_post_process.h 和 ak_nne_post_process.c。

11.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/yolox_nano_retrain/
../../Tools/Convert/convert    main_cfg_batch.txt
```

运行结束后在 nne_compile_tool/Demos/yolox_nano_retrain/目录下生成以下文件：

```
yolox_nano_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

Anyka Confidential For Giec Use Only

11.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/yolox_nano_retrain/
../../Tools/Quan/quan  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/yolox_nano_retrain/目录下生成以下文件：

```
yolox_nano_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

11.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/yolox_nano_retrain/
./vs_src/check  main_cfg_batch.txt
```

运行结束后，在 nne_compile_tool/Demos/yolox_nano_retrain/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt），保存了画框后的图片。

其他说明：

nne_compile_tool/Demos/yolox_nano_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

11.2.8 simulate

模拟开发板接口格式跑定点正向推理。

```
cd nne_compile_tool/Demos/yolox_nano_retrain/
./vs_src/simulate  main_cfg_batch.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 yolox_nano_input_416_416.rgbline，vs_simulate_output.txt。

其他说明：

nne_compile_tool/Demos/yolox_nano_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

11.2.9 编译

11.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

11.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目

Anyka Confidential For Internal Use Only

录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
yolox_nano_fixed_model.bin	nne_compile_tool/Demos/yolox_nano_retrain/ 或者 nne_compile_tool/上板验证/test_data/yolox_nano_retrain/
yolox_nano_input_416_416.rgbline	nne_compile_tool/Demos/yolox_nano_retrain/debug/ 或者 nne_compile_tool/上板验证/test_data/yolox_nano_retrain/

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo yolox_nano_retrain yolox_nano_fixed_model.bin yolox_nano_input_416_416.rgbline 416 416 3
```

打印结果与 nne_compile_tool/上板验证/test_data/yolox_nano_retrain/vs_simulate_output.txt 比对验证正确性。

12 Demo11: yolov5_n_retrain

12.1 来源

原始工程来自于 github 开源项目 [yolov5](https://github.com/ultralytics/yolov5) ，在此工程基础上修改网络结构以适应 AV100-NNE，并基于内部数据集重新训练，得到预训练模型，可检测人形、人脸。

12.2 步骤

12.2.1 准备测试/校准数据集

该 demo 未准备开源数据集，仅在 nne_compile_tool/Demos/yolov5_n_retrain/debug 目录下存放一张图片作为校准、测试图片，分辨率为 640*384，图片名信息保存在 nne_compile_tool/Demos/yolov5_n_retrain/debug/filename_list.txt。

12.2.2 填写配置文件

在 nne_compile_tool/Demos/yolov5_n_retrain/路径下已提供 main_cfg_one.txt 文件，内容填写如下：

base_dir	nne_compile_tool/Demos/yolov5_n_retrain/
onnx_file	yolov5n.onnx
input_w	640
input_h	384
input_c	3
input_color	NNE_INPUT_COLOR_RGB
input_norm	NNE_INPUT_NORM_ABS1
convert_float_model	yolov5n_float_model.bin
task	NNE_TASK_Detection
dataset_dir	nne_compile_tool/Demos/yolov5_n_retrain/debug/
dataset_list	nne_compile_tool/Demos/yolov5_n_retrain/debug/filename_list.txt

Anyka Confidential For Giec Use Only

quan_image_num	1
quan_method	NNE_QUAN_METHOD_Hist_Symmetry
quan_hist_th	0.001
quan_fixed_model	yolov5n_fixed_model.bin
test_float_en	AK_TRUE
test_quan_en	AK_TRUE
test_image_num	1
draw_result	AK_TRUE
image_bmp	debug/000000022935_resized.png
image_bin	debug/yolov5n_input_640_384.rgbline
pool_adjust	0
model_header	0 0 0 0

12.2.3 转换模型为 onnx

训练工程中提供 pytorch 转 onnx 的脚本。一般情况下 pytorch 转 onnx 可调用“torch.onnx.export”。如果生成的 onnx 包含网络后处理，可调用“onnx.utils.extract_model”提取 onnx 的子模型，将后处理部分的算子排除在外，参考《编程手册》3.1 节的说明。

12.2.4 写后处理 C 代码

该网络中对特征金字塔做处理得出最终目标框的过程作为后处理步骤。C 代码示例保存在 nne_compile_tool/Demos/yolov5_n_retrain/vs_src/目录下，ak_nne_post_process.h 和 ak_nne_post_process.c。

12.2.5 convert

该步骤将 onnx 模型转换为 AK 浮点模型。

运行命令行：

```
cd nne_compile_tool/Demos/yolov5_n_retrain/
../../Tools/Convert/convert    main_cfg_one.txt
```

运行结束后在 nne_compile_tool/Demos/yolov5_n_retrain/目录下生成以下文件：

```
yolov5n_float_model.bin
convert_middle_network.txt
convert_middle_param.bin
convert_output_net.txt
convert_output_net_analyse.txt
```

12.2.6 quan

该步骤生成 AK 定点模型。

运行命令行：

```
cd nne_compile_tool/Demos/yolov5_n_retrain/
../../Tools/Quan/quan    main_cfg_one.txt
```

运行结束后，在 nne_compile_tool/Demos/yolov5_n_retrain/目录下生成以下文件：

```
yolov5n_fixed_model.bin
quan_output_HistTH_feature_shift.txt
quan_output_HistTH_feature_shift_pos.txt
```

Anyka Confidential For Giec Use Only

```
quan_output_param_shift.txt
quan_output_param_feature_shift.txt
quan_output_network_info.c
quan_output_network_info.h
```

12.2.7 check

使用 Check 工程做正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/yolov5_n_retrain/
./vs_src/check   main_cfg_one.txt
```

运行结束后，在 nne_compile_tool/Demos/yolov5_n_retrain/目录下生成了两个文件夹 result_float 和 result_quan，分别存放浮点推理输出信息（result_float.txt）和定点推理输出信息（result_quan.txt），保存了画框后的图片。

其他说明：

nne_compile_tool/Demos/yolov5_n_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“check”部分的内容。

12.2.8 simulate

模拟开发板接口格式跑定点正向推理。

运行命令行：

```
cd nne_compile_tool/Demos/yolov5_n_retrain/
./vs_src/simulate   main_cfg_one.txt
```

运行结束后，在配置文件中 image_bin 指定路径生成 yolov5n_input_640_384.rgbline，vs_simulate_output.txt。

其他说明：

nne_compile_tool/Demos/yolov5_n_retrain/vs_src/目录下已包含编译所需文件及可执行文件。如果对文件做了修改则需重新编译。重新编译步骤参考《AnyCloud39AV100 NNE 网络移植编程手册》3.6.2 节“simulate”部分的内容。

12.2.9 编译

12.2.9.1 编译应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》准备好开发板环境并编译得到可执行文件：nne_demo。

12.2.9.2 运行应用 demo

参考《AnyCloud39AV100 NNE 网络移植编程手册》将下表中的文件放入开发板/mnt 目录。

文件名	来源
nne_demo	AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/
yolov5n_fixed_model.bin	nne_compile_tool/Demos/yolov5_n_retrain/ 或者 nne_compile_tool/上板验证/test_data/yolov5_n_retrain/
input_640_640.rgbline	nne_compile_tool/Demos/yolov5_n_retrain/debug/ 或者 nne_compile_tool/上板验证/test_data/yolov5_n_retrain/

Anyka Confidential For Gieo Use Only

进入 mnt 目录下

```
cd /mnt/
```

运行 nne_demo：

```
./nne_demo yolov5_n_retrain yolov5n_fixed_model.bin yolov5n_input_640_384.rgbline 640 384 3
```

打印结果与 nne_compile_tool/上板验证/test_data/yolov5_n_retrain/vs_simulate_output.txt 比对验证正确性。

Anyka Confidential For Giec Use Only